

CHƯƠNG 3. GIẢI PHÁP CHO BÀI TOÁN

3.1. Phát biểu bài toán

Hoạt động trao đổi thông tin trong doanh nghiệp thường được thực hiện qua các nền tảng nhắn tin thương mại hoặc dịch vụ đám mây của bên thứ ba. Các nền tảng này có ưu điểm về tính tiện dụng, tuy nhiên làm phát sinh một số vấn đề đối với tổ chức có nhu cầu quản lý dữ liệu nội bộ, kiểm soát tài khoản tập trung và tùy chỉnh hệ thống theo quy trình riêng. Dữ liệu hội thoại có thể nằm ngoài hạ tầng của doanh nghiệp; khả năng tích hợp với kho tri thức nội bộ bị giới hạn; chi phí sử dụng tăng theo số lượng tài khoản; và tổ chức phụ thuộc vào chính sách vận hành của nhà cung cấp.

Bên cạnh nhu cầu nhắn tin, nhân viên còn phải tra cứu nhiều tài liệu như quy định, hướng dẫn, chính sách nhân sự và cẩm nang nội bộ. Việc tìm kiếm thủ công mất thời gian, đặc biệt đối với nhân viên mới hoặc người chưa nắm rõ cấu trúc tài liệu. Vì vậy, bài toán đặt ra là xây dựng một hệ thống chat nội bộ có thể triển khai trên hạ tầng do tổ chức quản lý, đồng thời tích hợp trợ lý AI để hỗ trợ truy xuất và giải đáp thông tin từ nguồn tri thức nội bộ.

Giải pháp được xây dựng là hệ thống chat client-server theo kiến trúc microservices. Hệ thống cung cấp ứng dụng JavaFX cho người dùng, các dịch vụ backend độc lập, cơ chế xác thực tập trung bằng Keycloak, nhắn tin thời gian thực bằng WebSocket/STOMP, hệ thống thư điện tử nội bộ và dịch vụ AI sử dụng kỹ thuật Retrieval-Augmented Generation (RAG).

3.1.1. Đối tượng sử dụng

- **Doanh nghiệp và tổ chức:** triển khai hệ thống trên hạ tầng riêng, quản lý dữ liệu và tài khoản tập trung.
- **Nhân viên:** trao đổi thông tin qua hội thoại cá nhân, nhóm chat, tệp tin và hình ảnh.
- **Nhân viên mới:** sử dụng trợ lý AI để tiếp cận nhanh quy trình, chính sách và tài liệu nội bộ.
- **Bộ phận nhân sự:** cung cấp nguồn tri thức để AI hỗ trợ trả lời các câu hỏi thường gặp.
- **Người vận hành hệ thống:** cấu hình dịch vụ, theo dõi container, cơ sở dữ liệu, Keycloak và máy chủ thư.

3.1.2. Yêu cầu chức năng

Nhóm chức năng	Yêu cầu
Tài khoản	Đăng ký, đăng nhập, đăng xuất, làm mới phiên đăng nhập, cập nhật hồ sơ và đặt lại mật khẩu.
Thư điện tử nội bộ	Tạo mailbox khi đăng ký, cung cấp thông tin đăng nhập một lần, nhận thư đặt lại mật khẩu và truy cập webmail Roundcube.
Bạn bè	Thêm bạn, xóa bạn, tìm kiếm và hiển thị danh sách bạn bè.
Hội thoại	Tạo hội thoại cá nhân, tạo nhóm, thêm hoặc xóa thành viên, phân quyền thành viên, chuyển chủ nhóm và xóa nhóm.
Tin nhắn	Gửi và nhận tin nhắn thời gian thực, hiển thị trạng thái, thả cảm xúc, chỉnh sửa, thu hồi, xóa phía người dùng, đánh dấu sao và ghim.
Tệp đính kèm	Tải lên, gửi, nhận và tải xuống tệp hoặc hình ảnh trong hội thoại.
Trợ lý AI	Nhận câu hỏi, truy xuất tri thức nội bộ, xếp hạng tài liệu liên quan và sinh câu trả lời bằng mô hình ngôn ngữ chạy cục bộ.

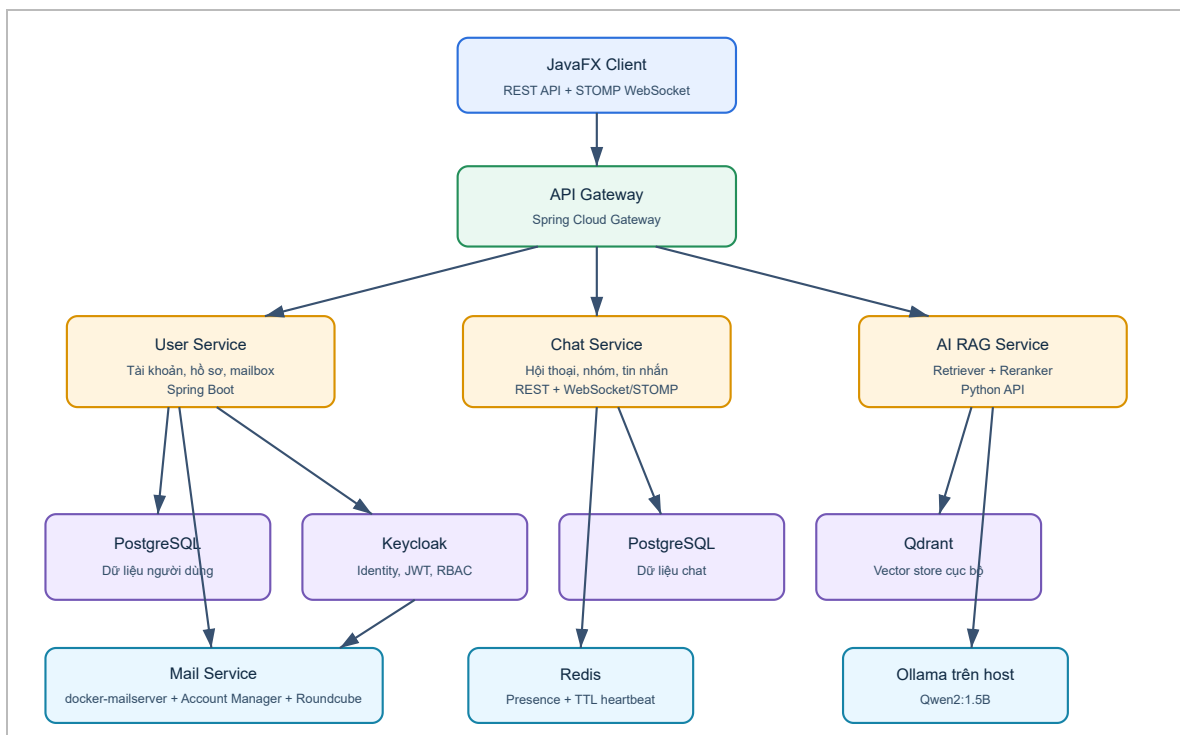
3.1.3. Yêu cầu phi chức năng

- **Bảo mật:** các API cần xác thực bằng JWT; mật khẩu người dùng do Keycloak quản lý; mật khẩu mailbox được lưu dưới dạng băm bởi hệ thống mail.
- **Riêng tư dữ liệu:** dữ liệu người dùng, hội thoại và tài liệu AI được lưu trên hạ tầng do tổ chức kiểm soát.
- **Thời gian thực:** tin nhắn và trạng thái nhập phải được đẩy đến client qua kết nối WebSocket thay vì polling liên tục.
- **Khả năng bảo trì:** các dịch vụ được phân tách theo miền nghiệp vụ; mã nguồn backend áp dụng phân lớp domain, application, infrastructure và API.
- **Khả năng mở rộng:** từng dịch vụ có thể được nâng cấp hoặc cấp thêm tài nguyên độc lập.
- **Khả năng triển khai:** các thành phần backend, cơ sở dữ liệu và mail được cấu hình thống nhất bằng Docker Compose.
- **Tính nhất quán:** việc đăng ký phải có cơ chế bù trừ khi tạo mailbox, tài khoản Keycloak hoặc bản ghi người dùng thất bại.

3.2. Thiết kế giải pháp

3.2.1. Kiến trúc tổng quan

Hệ thống sử dụng kiến trúc microservices, trong đó ứng dụng JavaFX giao tiếp với một điểm vào tập trung là API Gateway. Gateway định tuyến yêu cầu đến dịch vụ người dùng, dịch vụ chat và dịch vụ AI. Keycloak đảm nhiệm xác thực và phát hành JWT. Mỗi miền dữ liệu chính sử dụng cơ sở dữ liệu PostgreSQL riêng. Hệ thống mail cung cấp SMTP/IMAP và giao diện webmail. Các thành phần backend được triển khai bằng Docker Compose, trong khi Ollama hiện được chạy trên máy host và được AI Service truy cập qua REST API nội bộ.



Hình 3.1. Kiến trúc tổng quan của hệ thống chat nội bộ tích hợp trợ lý AI.

3.2.2. Vai trò của các thành phần

a) JavaFX Client

JavaFX Client cung cấp giao diện đăng ký, đăng nhập, quên mật khẩu, quản lý hồ sơ, danh sách bạn bè, hội thoại cá nhân, nhóm chat và trợ lý AI. Client sử dụng REST API cho các thao tác truy vấn hoặc cập nhật dữ liệu và duy trì kết nối STOMP WebSocket để gửi, nhận tin nhắn, trạng thái nhập và sự kiện thời gian thực. Phiên đăng nhập gồm access token và refresh token; client thực hiện làm mới token khi cần thiết và đưa người dùng về màn hình đăng nhập nếu phiên không còn hợp lệ.

b) API Gateway

API Gateway được xây dựng bằng Spring Cloud Gateway và cung cấp một địa chỉ truy cập thống nhất cho client. Gateway định tuyến các đường dẫn người dùng đến `user-service`, các đường dẫn hội thoại, nhóm, tin nhắn và bạn bè đến `chat-service`, đường dẫn AI đến `ai-rag-bot`, đồng thời chuyển tiếp kết nối `/ws` đến

Chat Service. Gateway được cấu hình như một OAuth2 Resource Server để kiểm tra JWT theo realm của Keycloak.

c) User Service

User Service được xây dựng bằng Spring Boot theo định hướng Clean Architecture. Lớp domain chứa mô hình người dùng và các value object; lớp application định nghĩa use case và port; lớp infrastructure hiện thực kết nối PostgreSQL, Keycloak và Mail Account Manager; lớp API cung cấp REST controller. Dịch vụ xử lý đăng ký, đăng nhập, làm mới token, đăng xuất, quên mật khẩu và cập nhật hồ sơ.

Khi đăng ký, User Service sinh địa chỉ email theo mẫu `username@gitlab.handbook.local` và mật khẩu mailbox ngẫu nhiên dài 16 ký tự. Dịch vụ tạo mailbox trước, tạo người dùng trong Keycloak, sau đó lưu hồ sơ vào PostgreSQL. Nếu một bước thất bại, các tài nguyên đã tạo trước đó được xóa để hạn chế dữ liệu không nhất quán.

d) Chat Service

Chat Service quản lý hồ sơ chat, quan hệ bạn bè, hội thoại, nhóm, thành viên nhóm, tin nhắn, reaction và tệp đính kèm. Dịch vụ sử dụng PostgreSQL và được tổ chức thành các lớp API, application, domain và infrastructure. REST API phục vụ các thao tác CRUD và tải lịch sử; STOMP WebSocket phục vụ gửi tin nhắn và trạng thái nhập theo thời gian thực. Simple Broker được cấu hình với các đích `/topic`, `/queue` và tiền tố người dùng `/user`.

e) AI RAG Service

AI Service được phát triển bằng Python và triển khai trong container `ai-rag-bot`. Dịch vụ nhận câu hỏi, truy xuất các đoạn tài liệu liên quan từ Qdrant, kiểm tra câu hỏi trùng khớp với tập hỏi đáp chuẩn, xếp hạng lại kết quả bằng reranker và tạo prompt gồm ngữ cảnh cùng câu hỏi. Prompt được gửi đến mô hình `Qwen2:1.5B` qua Ollama. Ollama hiện chạy trên máy host và được container truy cập bằng địa chỉ `host.docker.internal:11434`.

f) Keycloak

Keycloak quản lý thông tin định danh, mật khẩu đăng nhập, vai trò và phiên xác thực. Sau khi đăng nhập thành công, Keycloak phát hành access token và refresh token. Các dịch vụ xác minh chữ ký và issuer của JWT trước khi cho phép truy cập tài nguyên. Khi người dùng yêu cầu quên mật khẩu, User Service gọi Keycloak Admin API với action `UPDATE_PASSWORD`; Keycloak tạo liên kết đặt lại mật khẩu và gửi qua SMTP nội bộ.

g) Mail Service

Mail Service gồm `docker-mailserver`, `mail-account-manager`, Roundcube và cơ sở dữ liệu PostgreSQL của Roundcube. Docker-mailserver cung cấp SMTP, IMAP và lưu mailbox cùng mật khẩu dạng băm. Mail Account Manager cung cấp API nội bộ để User Service tạo hoặc xóa mailbox. Roundcube là giao diện webmail cho phép người dùng đăng nhập, đọc và gửi thư. Trong môi trường hiện tại, SMTP sử dụng cổng 587 với TLS và IMAP sử dụng cổng 993 với SSL.

h) Redis Presence

Redis lưu trạng thái trực tuyến bằng key có thời gian sống 75 giây. JavaFX Client gửi heartbeat đã xác thực bằng JWT mỗi 30 giây. Chat Service lấy định danh `sub` từ JWT, ánh xạ sang mã hồ sơ chat và gia hạn TTL. Khi heartbeat dừng, key tự hết hạn và người dùng được xác định là offline mà không cần tiến trình dọn dẹp riêng.

3.2.3. Kiến trúc nội bộ của các dịch vụ

User Service và Chat Service áp dụng nguyên tắc phân tách trách nhiệm theo Clean Architecture và Hexagonal Architecture. Chi tiết tên package có khác nhau giữa hai dịch vụ nhưng đều tuân theo hướng phụ thuộc từ ngoài vào trong.

- **Domain:** chứa entity, value object, enum, ngoại lệ nghiệp vụ và interface repository.
- **Application:** chứa use case, service điều phối nghiệp vụ, command, result và các port giao tiếp bên ngoài.
- **Infrastructure:** hiện thực repository, adapter Keycloak, adapter mail, HTTP client và cấu hình framework.
- **API:** tiếp nhận HTTP hoặc WebSocket request, kiểm tra dữ liệu đầu vào và chuyển kết quả sang DTO.

Cách tổ chức này giúp nghiệp vụ không phụ thuộc trực tiếp vào PostgreSQL, Keycloak hay docker-mailserver. Các tích hợp bên ngoài có thể thay đổi thông qua adapter mà ít ảnh hưởng đến use case cốt lõi.

3.2.4. Thiết kế giao tiếp

Hình thức	Mục đích	Thành phần
REST/JSON	Đăng ký, đăng nhập, quản lý hồ sơ, bạn bè, nhóm, lịch sử tin nhắn, tải tệp và gọi AI.	JavaFX, API Gateway và các microservice.
STOMP WebSocket	Gửi/nhận tin nhắn, trạng thái nhập và sự kiện thời gian thực.	JavaFX, API Gateway và Chat Service.
OAuth2/OpenID Connect	Đăng nhập, cấp access token, refresh token và xác minh JWT.	JavaFX, User Service, API Gateway và Keycloak.
SMTP/IMAP	Gửi thư đặt lại mật khẩu, đọc và gửi thư nội bộ.	Keycloak, docker-mailserver và Roundcube.
REST nội bộ	Quản lý mailbox và gọi mô hình sinh văn bản.	User Service với Mail Account Manager; AI Service với Ollama.

3.2.5. Luồng đăng ký tài khoản

1. Người dùng nhập tên đăng nhập, mật khẩu, họ tên và số điện thoại trên JavaFX Client.
2. Client gửi yêu cầu đăng ký qua API Gateway đến User Service.
3. User Service kiểm tra mật khẩu xác nhận, tên đăng nhập và email nội bộ đã tồn tại hay chưa.
4. Hệ thống tạo địa chỉ email nội bộ và sinh mật khẩu mailbox bằng bộ sinh số ngẫu nhiên an toàn.
5. User Service gọi Mail Account Manager để tạo mailbox trong docker-mailserver.
6. User Service gọi Keycloak Admin API để tạo tài khoản và thiết lập mật khẩu đăng nhập.
7. Hồ sơ người dùng được lưu vào PostgreSQL và liên kết với định danh Keycloak.
8. Client hiển thị một lần địa chỉ mailbox, mật khẩu mailbox và đường dẫn Roundcube để người dùng lưu lại.
9. Nếu bước tạo Keycloak hoặc lưu dữ liệu thất bại, hệ thống thực hiện bù trừ bằng cách xóa tài khoản hoặc mailbox đã tạo.

3.2.6. Luồng đăng nhập và làm mới phiên

1. Client gửi tên đăng nhập và mật khẩu đến User Service.
2. User Service sử dụng Keycloak Token API để xác thực.
3. Keycloak trả về access token, refresh token và thời gian hết hạn.
4. Client lưu thông tin phiên và gắn access token vào header `Authorization: Bearer`.
5. Khi access token hết hạn, client dùng refresh token để yêu cầu token mới.
6. Nếu refresh token không còn hợp lệ, client xóa phiên và yêu cầu người dùng đăng nhập lại.

3.2.7. Luồng quên mật khẩu

1. Người dùng nhập phần tên của địa chỉ email nội bộ; client ghép với miền `@gitlab.handbook.local`.
2. User Service tìm người dùng theo email nhưng phản hồi theo cách không tiết lộ email có tồn tại hay không.
3. Nếu tồn tại, User Service yêu cầu Keycloak gửi action email `UPDATE_PASSWORD`.
4. Keycloak tạo liên kết đặt lại mật khẩu với địa chỉ frontend có thể truy cập từ trình duyệt.
5. Keycloak gửi thư qua docker-mailserver; người dùng mở Roundcube để nhận liên kết.
6. Sau khi đổi mật khẩu, các phiên cũ có thể không còn hợp lệ; client thực hiện refresh hoặc yêu cầu đăng nhập lại.

3.2.8. Luồng gửi và nhận tin nhắn

1. Client mở hội thoại và tải lịch sử qua REST API.
2. Client duy trì kết nối STOMP WebSocket qua API Gateway.
3. Khi gửi tin nhắn, client tạo bản ghi tạm có trạng thái `SENDING` để phản hồi giao diện ngay lập tức.
4. Tin nhắn được gửi đến đích WebSocket của Chat Service.
5. Chat Service xử lý nghiệp vụ, lưu dữ liệu vào PostgreSQL và phát sự kiện đến người nhận.
6. Client đối chiếu tin nhắn phản hồi với bản ghi tạm, cập nhật mã tin nhắn và trạng thái.
7. Nếu hội thoại đang mở, tin nhắn được đánh dấu đã xem; nếu chưa mở, số lượng chưa đọc được tăng.
8. Các sự kiện chỉnh sửa, thu hồi, reaction hoặc ghim được đồng bộ vào danh sách đang hiển thị.

3.2.9. Luồng xử lý tệp đính kèm

1. Client kiểm tra tệp và giới hạn kích thước trước khi gửi.
2. Tệp được tải lên Chat Service bằng multipart request; giao diện hiển thị tiến trình tải.
3. Chat Service lưu tệp vào volume `chat_uploads` và trả về đường dẫn truy cập.
4. Client gửi metadata của tệp qua WebSocket như tên, kích thước, loại và URL.
5. Người nhận tải nội dung tệp thông qua endpoint của Chat Service.

Cấu hình hiện tại của Chat Service và API Gateway giới hạn multipart ở 50 MB. Vì vậy, giới hạn kiểm tra phía JavaFX cần được giữ bằng hoặc thấp hơn 50 MB để thống nhất toàn hệ thống.

3.2.10. Luồng cập nhật trạng thái trực tuyến

1. Sau khi đăng nhập, JavaFX khởi động bộ lập lịch heartbeat.
2. Mỗi 30 giây, client gửi yêu cầu đến `/api/presence/heartbeat` kèm access token.
3. API Gateway kiểm tra JWT và chuyển yêu cầu đến Chat Service.
4. Chat Service ánh xạ subject trong JWT sang mã người dùng nội bộ và ghi thời điểm hiện tại vào Redis.
5. Redis đặt TTL 75 giây; các heartbeat tiếp theo tiếp tục gia hạn TTL.
6. Khi không còn heartbeat, key hết hạn và trạng thái người dùng chuyển thành offline.

3.2.11. Luồng xử lý câu hỏi bằng RAG

1. Người dùng gửi câu hỏi đến hội thoại trợ lý AI.
2. AI Service chuẩn hóa câu hỏi và truy xuất các ứng viên liên quan từ Qdrant.
3. Hệ thống kiểm tra khả năng trùng khớp với tập câu hỏi - trả lời chuẩn.
4. Nếu không có kết quả trùng khớp đủ cao, reranker xếp hạng lại các đoạn tài liệu.
5. Các đoạn có độ liên quan cao nhất được ghép thành phần ngữ cảnh của prompt.
6. AI Service gửi prompt đến Qwen2:1.5B thông qua REST API của Ollama.
7. Câu trả lời và nguồn tham chiếu được ghi log và trả lại client.
8. Nếu không tìm thấy ngữ cảnh phù hợp, hệ thống trả lời rằng kho tri thức chưa có thông tin.

3.2.12. Thiết kế dữ liệu

Dữ liệu được phân chia theo miền nghiệp vụ để hạn chế phụ thuộc trực tiếp giữa các dịch vụ. User Service quản lý dữ liệu tài khoản ứng dụng; Chat Service quản lý dữ liệu giao tiếp; Keycloak quản lý định danh và thông tin xác thực; Roundcube sử dụng cơ sở dữ liệu riêng cho metadata webmail.

Kho dữ liệu	Dữ liệu chính	Thành phần sở hữu
User PostgreSQL	Hồ sơ người dùng, email nội bộ, trạng thái tài khoản và liên kết Keycloak.	User Service
Chat PostgreSQL	Hồ sơ chat, bạn bè, nhóm, thành viên, hội thoại, tin nhắn và reaction.	Chat Service
Keycloak database	Định danh, mật khẩu đăng nhập, role, phiên và token metadata.	Keycloak
Mail volumes	Mailbox, thư, trạng thái mail server và mật khẩu mailbox dạng băm.	docker-mailserver
Roundcube PostgreSQL	Thiết lập, metadata và dữ liệu phục vụ giao diện webmail.	Roundcube
Qdrant local store	Vector biểu diễn các đoạn tài liệu và tập hỏi đáp nội bộ.	AI Service
Redis	Trạng thái online và thời điểm heartbeat gần nhất với TTL.	Chat Service

3.2.13. Cơ chế bảo mật

Hệ thống sử dụng OAuth2/OpenID Connect và JWT làm nền tảng xác thực. JWT được ký bởi Keycloak; API Gateway và các resource server kiểm tra issuer cùng khóa công khai trước khi xử lý yêu cầu. Role trong token được sử dụng để hỗ trợ phân quyền. Mật khẩu đăng nhập không được lưu trong User Service mà do Keycloak quản lý.

- **Access token:** có thời hạn ngắn và được gửi kèm mỗi yêu cầu được bảo vệ.
- **Refresh token:** dùng để nhận access token mới mà không yêu cầu nhập lại mật khẩu.
- **WebSocket:** client truyền access token khi thiết lập kết nối STOMP.
- **Mail:** SMTP sử dụng TLS; IMAP sử dụng SSL; mailbox được tách khỏi mật khẩu chat.
- **Quên mật khẩu:** API không công khai việc email có tồn tại, giảm nguy cơ dò tài khoản.
- **Mạng Docker:** các dịch vụ giao tiếp bằng hostname nội bộ; mail sử dụng mạng riêng `mail-network`.
- **Dữ liệu nhạy cảm:** token, mật khẩu database và khóa nội bộ phải được cấu hình qua biến môi trường khi triển khai thực tế.

Các mật khẩu, token nội bộ và khóa bí mật được cung cấp qua tệp `.env` cục bộ đã loại khỏi Git; Docker Compose từ chối khởi động nếu thiếu các biến bắt buộc. PostgreSQL và Redis chỉ được truy cập trong mạng Docker, không công khai cổng ra host. API Gateway, Keycloak, Roundcube, SMTP Submission và IMAPS sử dụng TLS với certificate được mount từ thư mục `secrets`; private key và certificate không được lưu trong repository.

3.3. Hiện thực giải pháp

3.3.1. Hiện thực JavaFX Client

Client được phát triển bằng Java 21, JavaFX và FXML. Các màn hình chính gồm đăng nhập, đăng ký, quên mật khẩu, thông tin mailbox, giao diện chat và hồ sơ người dùng. Mô hình MVVM được áp dụng để tách giao diện khỏi nghiệp vụ phía client. `ApiClient` quản lý giao tiếp REST và phiên đăng nhập; `RealtimeChatService` quản lý STOMP WebSocket.

Phần chat được chia thành các thành phần chuyên trách như quản lý danh bạ, hội thoại, gửi tin nhắn, realtime, nhóm, hồ sơ và thao tác tin nhắn. Cách phân tách này giảm kích thước của `ChatViewModel`, hạn chế phụ thuộc chéo và tạo điều kiện kiểm thử từng workflow.

3.3.2. Hiện thực User Service

User Service cung cấp các endpoint xác thực và quản lý người dùng. Các controller chỉ tiếp nhận DTO và gọi use case. `AuthApplicationService` điều phối đăng ký, đăng nhập, refresh token, đăng xuất và quên mật khẩu. `KeycloakUserAdapter` cùng `KeycloakTokenAdapter` đóng gói Keycloak Admin API và Token API. `DockerMailserverMailboxAdapter` hiện thực port quản lý mailbox.

3.3.3. Hiện thực Chat Service

Chat Service cung cấp controller cho hội thoại, nhóm, bạn bè, tin nhắn và hồ sơ chat. Mỗi miền nghiệp vụ có use case và application service riêng. Các repository adapter sử dụng Spring Data JPA để truy cập PostgreSQL. WebSocket được cấu hình bằng `@EnableWebSocketMessageBroker`; controller WebSocket xử lý đích gửi tin nhắn và trạng thái nhập.

3.3.4. Hiện thực AI Service

Pipeline RAG gồm bốn bước chính: truy xuất, kiểm tra QA trùng khớp, reranking và sinh câu trả lời. Hệ thống sử dụng mô hình embedding `BAAI/bge-m3`, mô hình reranker `BAAI/bge-reranker-base` và mô hình sinh `qwen2:1.5b`. Kết quả xử lý được lưu vào log JSONL, bao gồm câu hỏi, câu trả lời, nguồn và điểm truy xuất để hỗ trợ đánh giá hoặc truy vết.

3.3.5. Hiện thực Mail Service

Docker-mailserver được cấu hình theo cơ chế account provisioner dạng file và lưu dữ liệu qua Docker volume. Mail Account Manager gọi lệnh quản trị của docker-

mailserver thông qua Docker socket và bảo vệ API bằng token nội bộ. Roundcube kết nối đến IMAP cổng 993 và SMTP cổng 587, đồng thời sử dụng PostgreSQL để lưu metadata của webmail.

3.3.6. Triển khai bằng Docker Compose

Tệp `chat-system/docker-compose.yml` mô tả môi trường tích hợp của hệ thống. Các container chính gồm:

- `postgres`, `keycloak` và `user-service` cho miền người dùng.
- `chat-postgres` và `chat-service` cho miền chat.
- `redis` lưu trạng thái online/offline theo TTL.
- `ai-rag-bot` cho dịch vụ AI; Ollama chạy trên host.
- `api-gateway` làm điểm vào HTTP và WebSocket.
- `mailserver`, `mail-account-manager`, `roundcube` và `roundcube-db` cho hệ thống thư.

Docker volume được sử dụng để duy trì dữ liệu PostgreSQL, tệp chat, mailbox, trạng thái mail và dữ liệu Roundcube sau khi container được khởi động lại. Thứ tự phụ thuộc và healthcheck giúp các dịch vụ chỉ khởi động khi thành phần cần thiết đã sẵn sàng.

3.3.7. Cấu hình cổng dịch vụ trong môi trường phát triển

Thành phần	Cổng host	Mục đích
Keycloak	8081	HTTPS đăng nhập, quản trị realm và xử lý action token.
User Service	Không công khai	API tài khoản và hồ sơ trong mạng Docker.
Chat Service	Không công khai	API chat và WebSocket trong mạng Docker.
Roundcube	8085	Giao diện webmail qua HTTPS.
API Gateway	8088	Điểm vào HTTPS/WSS chung cho client.
AI RAG Service	Không công khai	API trợ lý AI được truy cập qua Gateway.
Redis	Không công khai	Presence online/offline và TTL heartbeat trong mạng Docker.
Ollama	11434	REST API mô hình ngôn ngữ trên host.
SMTP Submission	587	Gửi thư qua STARTTLS.
IMAPS	993	Truy cập hộp thư qua TLS.

3.3.8. Kiểm thử và xác minh

Hệ thống được kiểm tra ở mức unit test và kiểm tra tích hợp trong môi trường Docker. Phía JavaFX có kiểm thử cho view model xác thực, parser emoji, tiện ích liên kết, dịch vụ realtime và trạng thái hiển thị tin nhắn. Các kịch bản tích hợp cần xác minh gồm đăng ký đồng thời tạo mailbox, đăng nhập và refresh token, gửi

email đặt lại mật khẩu, gửi nhận tin nhắn WebSocket, quản lý nhóm, tải tệp và gọi trợ lý AI.

3.4. Đánh giá giải pháp

3.4.1. Kết quả đạt được

- Xây dựng được ứng dụng chat JavaFX kết nối với hệ thống backend nhiều dịch vụ.
- Tách biệt miền người dùng, chat, AI và mail, giúp quản lý trách nhiệm rõ ràng.
- Tích hợp Keycloak để quản lý danh tính, JWT, refresh token và đặt lại mật khẩu.
- Hỗ trợ nhắn tin thời gian thực, quản lý nhóm, bạn bè, tệp, reaction, ghim và lịch sử.
- Tạo tự động mailbox nội bộ và cung cấp giao diện Roundcube cho người dùng.
- Tích hợp pipeline RAG và mô hình Qwen chạy cục bộ để hỗ trợ hỏi đáp nội bộ.
- Sử dụng Redis heartbeat và TTL để quản lý trạng thái online/offline.
- Đóng gói phần lớn thành phần bằng Docker Compose, thuận tiện dựng môi trường thử nghiệm.

3.4.2. Hạn chế

- Môi trường hiện tại vẫn sử dụng một số mật khẩu và chứng chỉ tự ký dành cho phát triển.
- Ollama chạy trên host nên toàn bộ hệ thống chưa được đóng gói hoàn toàn trong Docker Compose.
- Simple Broker của Spring phù hợp quy mô thử nghiệm nhưng chưa tối ưu cho triển khai nhiều instance.
- Hệ thống chưa có cơ chế giám sát tập trung, distributed tracing và tổng hợp log.
- Admin Dashboard chưa có phần hiển thị đầy đủ trong mã nguồn hiện tại.
- Giới hạn tệp giữa JavaFX và backend cần được cấu hình thống nhất.

3.4.3. Hướng cải tiến

- Đưa secret vào Docker Secrets hoặc hệ thống quản lý bí mật chuyên dụng.
- Sử dụng HTTPS/WSS với chứng chỉ hợp lệ cho toàn bộ luồng bên ngoài.
- Đóng gói Ollama hoặc cấu hình GPU server riêng cho môi trường sản xuất.
- Thay Simple Broker bằng RabbitMQ hoặc broker hỗ trợ STOMP khi mở rộng nhiều Chat Service.
- Bổ sung OpenTelemetry, distributed tracing và hệ thống log tập trung.
- Bổ sung rate limiting, antivirus cho tệp tải lên, sao lưu và chính sách lưu giữ dữ liệu.
- Hoàn thiện Admin Dashboard để quản lý tài khoản, vai trò và trạng thái hệ thống.
- Xây dựng bộ đánh giá chất lượng RAG dựa trên độ chính xác, độ liên quan và thời gian phản hồi.

3.5. Kết luận chương

Chương này đã trình bày giải pháp cho bài toán xây dựng hệ thống chat nội bộ tích hợp trợ lý AI, từ yêu cầu chức năng, kiến trúc tổng quan, thiết kế giao tiếp, cơ chế bảo mật đến cách hiện thực và triển khai. Giải pháp kết hợp JavaFX, Spring Boot, Spring Cloud Gateway, Keycloak, PostgreSQL, WebSocket/STOMP, Docker, hệ thống mail nội bộ và pipeline RAG sử dụng Qwen qua Ollama. Kiến trúc phân tách theo miền nghiệp vụ giúp hệ thống dễ bảo trì, có khả năng mở rộng và phù hợp với mục tiêu kiểm soát dữ liệu trên hạ tầng riêng của tổ chức.